

TreeMAPPO: Fine-Grained Credit Assignment for Agentic LLM Training via Token-Level Trajectory Branching

Anonymous submission

Abstract

We present TreeMAPPO, a reference-free framework for fine-grained credit assignment in agentic large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMAPPO constructs a rollout tree, estimates segment-level contribution and uncertainty from mixed-success leaves, and converts these estimates into dense token-level training signals. The method is designed for mathematical reasoning agents whose trajectories may interleave natural-language reasoning, fenced Python tool calls, tool responses, and final boxed answers. Our implementation combines guided token-level branching, a maximum-a-posteriori posterior model over segment contributions, greedy trajectory de-duplication for training-set construction, and advantage-weighted causal language-model updates. [We expect TreeMAPPO to outperform trajectory-level baselines such as GRPO, especially when sparse outcome rewards obscure which intermediate reasoning decisions were beneficial or harmful.](#)

Introduction

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning agents. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute in agentic settings, where early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained agentic branching methods (Tran, Yao, and Yu 2025; Yang et al. 2025; Li et al. 2025; Huang, Nguyen, and Zimmer 2025; Wang et al. 2026). These results suggest that branching structure can expose more informative local comparisons than flat rollout groups. At the same time, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper studies fine-grained credit assignment for mathematical reasoning agents through token-level trajectory branching. Our central idea is to turn a small set of

full rollouts into a structured tree of alternative continuations, then infer which segments were likely helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMAPPO (Tree-based Maximum-A-Posteriori Policy Optimization).

Our implementation uses a lightweight tool-calling protocol: the model receives a question and an optional Python tool specification, emits reasoning in text, may invoke the tool via fenced Python markdown blocks, observes the tool response in context, and is required to terminate with a final answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior estimates are transformed into token-level training advantages.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes uncertain segments and branching nodes while discouraging over-fragmentation and over-concentration.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We provide an experimental framework spanning tool and no-tool reasoning, in-distribution and out-of-distribution mathematical benchmarks, and ablations that separate branching strategy from credit estimation.

[Blue text marks hypotheses, placeholders, or details that should be replaced once final experimental results are available.](#)

Related Work

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards has become a standard recipe for improving mathematical and agentic reasoning, but trajectory-level objectives still face a severe credit-assignment problem. Actor-critic methods such as PPO provide token-level advantages through a learned value function (Schulman et al. 2017), while critic-free group-relative methods simplify training but typically distribute supervision at the trajectory level. This tension has motivated a re-

cent wave of work on structured rollouts and finer-grained credit.

Implicit tree and shared-prefix methods. TEMPO (Tran, Yao, and Yu 2025) converts a group of sampled responses into a prefix tree and adds branch-gated temporal-difference corrections on top of a GRPO-style training loop. Its main appeal is that it preserves a relatively simple rollout interface while extracting token-level signal where natural branching occurs. Our method differs in that we do not rely on incidental prefix overlap alone; instead, we explicitly create new branches at selected token positions, which provides direct control over branching density and lets exploration be driven by posterior uncertainty.

Explicit tree sampling and tree-aware advantages. TreeRPO (Yang et al. 2025) and TreePO (Li et al. 2025) both pursue explicit tree-structured sampling to obtain denser supervision. TreeRPO estimates step-level reward expectations via tree sampling and forms step-level comparison groups without a separate reward model. TreePO emphasizes systems efficiency, shared-prefix reuse, and subgroup-relative advantages over segment-level trees. Our method is closest in spirit to this line, but differs in two important respects: first, we branch at arbitrary token positions rather than imposing fixed step or segment partitions; second, we infer segment contribution with a probabilistic MAP model that explicitly represents uncertainty, rather than relying only on win-rate or subgroup heuristics.

Prefix-aware and off-policy tree supervision. Tree-OPO (Huang, Nguyen, and Zimmer 2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMAPPO is on-policy and reference-free: the tree is constructed directly from the policy’s own rollouts, without offline teacher trees, reward models, or process-level supervision.

Agentic RL and branching granularity. APPO (Wang et al. 2026) studies branching and credit assignment in tool-using agents, arguing that influential decision points are not confined to tool-call boundaries or coarse workflow stages. This observation aligns closely with our setting. The main methodological difference is that APPO uses a branching score and procedure-level advantage scaling, whereas TreeMAPPO combines explicit token-level branching with posterior inference over signed segment contribution and uncertainty.

Problem Setting and Agent Protocol

We study question-answering agents for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

Our implementation uses the following protocol.

1. Each prompt contains the problem statement and, when tool use is enabled, a specification that Python can be called by emitting a fenced markdown block.

2. The model may interleave reasoning and tool use.
3. When a fenced Python block closes, generation pauses, the tool executes, and the tool response is appended to the context.
4. The model is instructed to present its final answer in `\boxed{}`.
5. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

This protocol is consistent across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes.

Method

Overview

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into branching scores for future expansions and into training advantages for model optimization.

Trajectory Tree Construction

In the default configuration for our main tool-using setup, we begin with 4 trunk trajectories and expand to 16 total trajectories. Our experimental plan also includes branch-budget variants with 8 and 32 total trajectories.

Branching is token-level rather than step-level. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

The implementation keeps a fixed decoding temperature within each tree. In the current configs, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

MAP Credit Assignment Model

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ :

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ .
- Segment i has mean contribution m_i and log standard deviation u_i .

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \quad (1)$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \quad (2)$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \quad (3)$$

We use a probit likelihood,

$$p(y_\ell | \{m_i, u_i\}) = \Phi(z_\ell), \quad (4)$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = - \sum_\ell \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2} \sum_i m_i^2 + \frac{1}{2\sigma_u^2} \sum_i u_i^2. \quad (5)$$

The current hyperparameter file uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. Optimization is performed with gradient steps and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins.

Uncertainty-Aware Guided Branching

Posterior statistics are reused to decide where to branch next. The implementation first computes a signal-to-noise ratio for each segment,

$$r_i = \frac{m_i}{\exp(u_i) + \varepsilon}, \quad (6)$$

then standardizes these values across segments in the same tree,

$$\tilde{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \varepsilon}, \quad (7)$$

and maps them to a bounded uncertainty score,

$$s_i = \exp(-\alpha \tilde{r}_i^2), \quad \alpha = 1. \quad (8)$$

Segments whose posterior signal is close to zero receive higher branching priority because they are more uncertain.

For a branching node with parent score s_p and child scores $s_{c_1}, \dots, s_{c_b}$, the node score is

$$s_{\text{node}} = \frac{bs_p + \sum_{j=1}^b s_{c_j}}{2b}. \quad (9)$$

The final implementation-level per-token branching score also includes a model-side alternative-token term derived from the stored top- k logprob candidates. Concretely, for each candidate token position we multiply:

1. a segment or node uncertainty score,
2. a structural penalty, and
3. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp(-k_{\text{node}}(b-1)), \quad (10)$$

with $k_{\text{node}} = 0.35$ in the current implementation. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

From Posterior Scores to Training Advantages

After tree construction, each segment receives an unnormalized signed score proportional to $m_i / \exp(u_i)$. The methodology notes propose an additional length-aware rescaling to reduce volatility from very short reasoning spans. The generated training samples then attach token-level advantage values aligned with the input sequence.

The downstream Python training code consumes four aligned tensors per sample: `input_ids`, `labels`, `attention_mask`, and `advantages`. The loss is an advantage-weighted causal LM objective over supervised positions,

$$\mathcal{L}_{\text{train}} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \text{clip}(A_t, -c, c) \cdot \text{CE}(p_\theta(\cdot | x_{<t}), x_t), \quad (11)$$

where $\mathcal{S}$ denotes unmasked token positions and the current implementation clips A_t to $[-3, 3]$ before weighting the token cross-entropy. This is the implemented optimization interface for injecting segment-level credit into model updates.

Training-Set Construction

Because many tree leaves share prefixes, naively flattening all leaves would over-train shared segments. We therefore use a greedy de-duplication policy.

1. Score each candidate leaf trajectory by average absolute token advantage.
2. Select the strongest candidate.
3. Mark its segments as consumed.
4. Repeat among trajectories that still contain unconsumed learnable segments.
5. Aggregate selected trajectories across trees and prune the tail with a cumulative average-absolute-advantage cutoff.

Selected trajectories are finally ordered by sequence length for batching efficiency.

Baselines and Ablations

We consider three primary comparison points.

- **Base model:** no fine-tuning.
- **GRPO-style baseline:** use 8 trunks and 8 total trajectories, yielding multiple complete rollouts without token-level branching in the main tool setup.
- **TreeMAPPO:** guided branching with MAP-based posterior credit assignment.

We also separate two ablation axes that are explicitly encoded in configs and scripts.

- **TEMPO-style branching ablation:** switch the branching policy from guided branching to spontaneous divergence-point branching while keeping posterior-based credit assignment.
- **TreeRPO ablation:** keep guided branching but replace posterior-based segment advantages with a win-rate heuristic.

In our implementation, these ablations are not merely renamed conditions: the TEMPO ablation changes `branching_policy`, while the TreeRPO ablation changes `advantage_calculation_policy`. This distinction is important because it cleanly separates exploration structure from credit estimation.

Experimental Setup

Models

The current experimental plan covers the following model-condition families.

- Qwen2.5-7B-Instruct with tool use.
- Qwen2.5-7B-Instruct without tool use.
- Qwen3-4B with tool use.
- Qwen3-4B without tool use.
- Gemma-3-4B-IT without tool use.
- Llama-3.1 and Mistral-7B without tool use.

For the main tables, tool-enabled and no-tool variants of the same backbone should be treated as separate model-condition groups, since the prompting interface and action space differ.

Datasets

Training and validation are built from DeepMath, MATH, and GSM8K.

- `hybrid_train.sqlite`: 5,000 samples per dataset, interleaved across the three in-distribution datasets.
- `hybrid_val.sqlite`: 1,000 validation samples per dataset drawn from disjoint index ranges after the training slice.

We also construct an aggregated held-out evaluation database from six datasets in this order: DeepMath, MATH, GSM8K, AIME24, AIME25, and AMC23. For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the current pipeline, evaluation rows are drawn from the training split after clearing the 6,000 training-plus-validation examples reserved for in-distribution use.

Note on a current draft mismatch: the implementation presently builds 6 evaluation datasets (3 in-distribution + 3 out-of-distribution), whereas the reporting note mentions 7 columns with 4 out-of-distribution datasets. The table templates below follow the implemented 6-dataset setup and can be extended by one extra OOD column if an additional benchmark is added later.

Training and Evaluation Protocol

An orchestration epoch consists of four stages: validation rollout, training rollout collection, training-set generation, and training. Across epochs, the dataset is regenerated using the model produced by the previous epoch.

The current training plan uses the following recurring settings:

- 10 orchestration epochs.
- cumulative average-absolute-advantage cutoff of 0.5 for training-set retention.
- 3 optimizer iterations per training run.
- LoRA-based training configuration in the default scripts.
- fixed rollout temperature 0.7.

Insert the final production hardware description here. Current notes mention A100-based LoRA training, while example scripts expose a configurable GPU count and may be used both for smoke runs and full experiments.

The primary evaluation metric is pass@1 accuracy. We plan to report mean and confidence intervals over multiple independent runs.

Our expectation is that gains will be strongest on problems whose success depends on a few decisive reasoning turns rather than uniformly smooth reasoning quality across the whole trajectory.

Results

Since the full experimental sweep is still in progress, we present the reporting structure below with blue placeholders for measurements and forward-looking claims. These placeholders should be replaced with final statistics before submission.

Main Comparison

Table 1 is structured to compare base, GRPO, and TreeMAPPO across datasets, while grouping rows by model-condition family.

Expected findings. Across most model families, TreeMAPPO should improve over both the unfine-tuned baseline and GRPO, indicating that denser segment-level credit is more informative than trajectory-level reward alone. We expect the largest gains when tool use is available, because branching can better isolate whether a local reasoning segment or a tool-mediated correction changed the final outcome.

Ablation Study

The main ablation table is designed around the conditions explicitly implemented for Qwen2.5-7B with tool use: Base, GRPO, TEMPO-style spontaneous branching, TreeRPO-style heuristic credit assignment, and TreeMAPPO.

Branching-Budget Sensitivity

Our experimental plan already includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B tool setting. This enables a direct study of how tree size affects downstream performance.

Model family	Setup	DeepMath	MATH	GSM8K	AIME24	AIME25	AMC23	Avg.
Qwen2.5-7B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 1: Main comparison across model families and datasets. Replace blue placeholders with measured pass@1 mean $\pm$ confidence interval. The intended trend is Base < GRPO < TreeMAPPO in most rows, with especially visible gains for the tool-enabled Qwen settings.

Model family	Setup	DeepMath	MATH	GSM8K	AIME24	AIME25	AMC23	Avg.
Qwen2.5-7B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TEMPO-style	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeRPO-style	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 2: Ablation framework. The TEMPO-style condition changes the branching policy; the TreeRPO-style condition changes the segment-credit estimator. We expect TreeMAPPO to outperform both ablations, supporting the claim that guided branching and probabilistic credit assignment contribute complementary benefits.

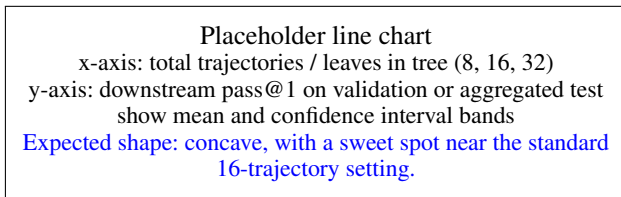


Figure 1: Branching-budget sensitivity. Replace this placeholder with a real line chart once results are available.

Expected findings. We hypothesize that too few leaves under-explore decisive alternative reasoning paths, while too many leaves dilute collection budget across marginal branches and introduce noisier training targets. This should yield a concave performance curve with an intermediate optimum.

Discussion

The main conceptual benefit of TreeMAPPO is that it turns sparse trajectory-level outcomes into dense supervision without requiring reference solutions or manually annotated intermediate steps. The tree structure also allows uncertainty to be localized: early segments supported by many mixed descendants can be recognized as uncertain, whereas a split that sharply changes success rates can receive stronger credit or blame.

Several practical design choices are worth emphasizing.

- Branch selection is not based on posterior uncertainty alone; it also depends on the model’s alternative-token probability mass, which makes branching more realistic and less likely to chase implausible continuations.
- The training loss is an advantage-weighted language-model loss rather than a clipped PPO-ratio objective. This improves engineering simplicity and stability, but the name *TreeMAPPO* should be explained carefully so

readers understand what is implemented.

- Tool-enabled and no-tool settings should be analyzed separately because the credit-assignment problem differs materially once trajectories can call Python and recover from arithmetic mistakes.

Limitations and Future Work

The present system has several important limitations.

- The current repository-aligned evaluation framework is broader than the set of finalized results.
- Some methodological notes discuss variants not fully surfaced in the default scripts, such as richer temperature calibration or additional model families.
- The present dataset builders and evaluation utilities should be audited before final reporting to ensure that every split and benchmark exactly matches the intended paper protocol.

Future work may explore adaptive branch temperatures, better posterior priors, richer tool environments, and branch-selection objectives that directly optimize information gain rather than uncertainty heuristics alone.

Conclusion

We presented TreeMAPPO, a trajectory-tree method for fine-grained credit assignment in agentic LLM training. The core idea is to branch reasoning trajectories at token granularity, infer segment-level contribution and uncertainty from mixed-success descendants, and convert those estimates into training signals for subsequent optimization. The current implementation already supports the central methodological ingredients, several ablations, tool/no-tool variants, and branch-budget experiments. [If the final experiments confirm the expected pattern, the results will support the claim that explicit tree structure and uncertainty-aware segment credit can improve both training efficiency and pass@1 accuracy over trajectory-level baselines.](#)

References

- Huang, B.; Nguyen, T.; and Zimmer, M. 2025. Tree-OPO: Off-policy Monte Carlo Tree-Guided Advantage Optimization for Multistep Reasoning. ArXiv preprint.
- Li, Y.; Gu, Q.; Wen, Z.; Li, Z.; Xing, T.; Guo, S.; Zheng, T.; Zhou, X.; Qu, X.; Zhou, W.; Zhang, Z.; Shen, W.; Liu, Q.; Lin, C.; Yang, J.; Zhang, G.; and Huang, W. 2025. TreePO: Bridging the Gap of Policy Optimization and Efficacy and Inference Efficiency with Heuristic Tree-based Modeling. arXiv:2508.17445.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Tran, H.; Yao, Z.; and Yu, H. 2025. Exploiting Tree Structure for Credit Assignment in RL Training of LLMs. TEMPO, arXiv:2509.18314.
- Wang, X.; Ma, Z.; Wang, Y.; Ji, Y.; Yang, S.; Chen, G.; Wang, P.; and Chu, X. 2026. APPO: Agentic Procedural Policy Optimization. arXiv:2606.12384.

Yang, Z.; Guo, Z.; Huang, Y.; Liang, X.; Wang, Y.; and Tang, J. 2025. TreeRPO: Tree Relative Policy Optimization. arXiv:2506.05183.